

read

קריאת המערכת `read()` היא הפעולה הבסיסית שבאמצעותה תוכנית במרחב המשתמש (User Space) שואבת נתונים מקובץ פתוח. במסגרת עבודה מול **Linux Kernel 6.14**, קריאה זו מתבצעת דרך שכבת ה-VFS, עושה שימוש נרחב במטמון הזיכרון (Page Cache) של הקרנל, ומנהלת את התקשורת עם התקני ה-SSD במידת הצורך.

להלן האלגוריתם המלא והמפורט של קריאת המערכת `read`, מתחילת הפנייה ועד החזרת הנתונים לתהליך.

אלגוריתם ה- `read()` – שלב אחר שלב

1. **שלב כניסה לקרנל ואיתור אובייקט הקובץ (`fdget / sys_read`):** מעבר ל- Ring 0 ומיפוי ה-FD (Descriptor) אל אובייקט הקובץ הפתוח בזיכרון (`struct file`).
2. **שלב בדיקת תקינות והרשאות קריאה (`File Status & f_mode`):** אימות שהקובץ אכן נפתח במצב המאפשר קריאה ושהנסיבות תקינות.
3. **שלב הפעלת ממשק ה-VFS (`new_sync_read / vfs_read`):** פנייה לשכבת ההפשטה של מערכות הקבצים והפעלת מתודת הקריאה המתאימה (`read_iter`).
4. **שלב בדיקת מטמון הזיכרון (`Page Cache Lookup - Hit vs. Miss`):** בדיקה האם הנתונים כבר קיימים בזיכרון ה-RAM (Page Cache) או שנדרשת קריאה פיזית מהדיסק.
5. **שלב קריאת הבלוקים מהדיסק (`SSD Read & Block I/O`) - במקרה של Cache Miss:** שליחת בקשה דרך שכבת הבלוקים ומנהל ההתקן של ה-NVMe לשליפת הנתונים מה-SSD.
6. **שלב העתקת הנתונים למרחב המשתמש (`copy_to_user`):** העברה בטוחה של הנתונים מבאפר הליבה אל תוך באפר הזיכרון של התהליך.
7. **שלב עדכון מיקום הקובץ (`f_pos`):** קידום מצביע הקריאה (`File Offset`) בהתאם למספר הבייטים שנקראו בפועל.
8. **שלב סיום והחזרה למרחב המשתמש:** שחרור הפניות, חזרה ל- Ring 3 והחזרת מספר הבייטים שנקראו.

הסבר מלא ומפורט לכל צעד באלגוריתם

- שלב 1: כניסה לקרנל ואיתור אובייקט הקובץ (fdget / sys_read)
 - מה קורה בפועל: תוכנית במרחב המשתמש קוראת לפונקציה:

```
ssize_t read(int fd, void *buf, size_t count);
```

פקודת ה-syscall מעבירה את המעבד מ-Ring 3 ל-Ring 0, והבקשה מגיעה לפונקציית הליבה sys_read.

- איתור ה-File Descriptor:** הקרנל מקבל את המספר השלם fd (תיאור הקובץ, לדוגמה 3). הוא פונה אל טבלת תיאורי הקבצים של התהליך (files<-current) כדי לשלוף את המצביע אל אובייקט הקישור struct file * התואם. אם ה-fd אינו תקין או שאינו פתוח, הקרנל עוצר מיד ומחזיר שגיאת EBADF (Bad file descriptor).

שלב 2: בדיקת תקינות והרשאות קריאה (f_mode)

- מה קורה בפועל:** הקרנל בודק את שדה ההרשאות באובייקט ה-struct file (שדה f_mode). הוא מוודא שהקובץ נפתח במקור עם דגל קריאה חוקי (כגון O_RDONLY או O_RDWR).

- אם הקובץ נפתח אך ורק לכתיבה (למשל עם O_WRONLY), הקרנל חוסם את הפעולה מיד ומחזיר שגיאת EBADF, משום שאין לו אישור לבצע עליו פעולת קריאה.

שלב 3: הפעלת ממשק ה-VFS (read_iter / vfs_read)

- מה קורה בפועל:** הקרנל מפעיל את פונקציית התווך הגנרית של ה-VFS בשם vfs_read.
- פונקציה זו בודקת האם הקובץ תומך בפעולות קריאה אסינכרוניות או סינכרוניות, וניגשת אל מערך הפעולות של מערכת הקבצים (read<-f_op או file<-f_op). עבור מערכת קבצים כמו ext4, הקריאה מנותבת למתודה הייעודית של .ext4_file_read_iter.

שלב 4: בדיקת מטמון הזיכרון (Page Cache Lookup - Hit vs. Miss)

זהו אחד השלבים החשובים ביותר בביצועי לינוקס:

- עיקרון ה-Page Cache:** ליבת לינוקס שומרת את כל הנתונים שנקראו מהדיסק בתוך אזור זיכרון ייעודי ב-RAM בשם Page Cache, המנוהל ברמת ה-Inode של הקובץ.
- חיפוש נתונים (Cache Lookup):** הקרנל מחשב איזה עמוד זיכרון (Page, בגודל KB4) מכיל את הבייטים המבוקשים בהתאם למיקום הנוכחי בקובץ (f_pos), ובודק בטבלת הזיכרון האם עמוד זה כבר קיים ב-RAM.

- Cache Hit (פגיעה במטמון):** הנתונים כבר קיימים בזיכרון ה-RAM (הובאו בעבר או נכתבו לאחרונה). הקרנל מדלג לחלוטין על הדיסק הפיזי וגש ישירות לזיכרון ה-RAM, מה שמעניק מהירות אדירה.
- Cache Miss (החטאה במטמון):** הנתונים אינם קיימים בזיכרון ה-RAM, ולכן הקרנל חייב לפנות אל התקן הפיזי (ה-SSD).

שלב 5: קריאת הבלוקים מהדיסק (SSD Read & Block I/O - ב-Cache Miss)

- **מה קורה בפועל: במקרה של Cache Miss :**

1. דרייבר מערכת הקבצים מתרגם את הדרישה הלוגית לבלוקים פיזיים על גבי ה-SSD (תוך שימוש בעץ ה-Extents של ה-Inode).
2. הקרנל מייצר אובייקט bio ושולח בקשת I/O דרך שכבת הבלוקים (blk-mq) ובקר ה-NVMe.
3. הבקר קורא את הבלוקים הפיזיים מה-SSD אל תוך ה-Page Cache שהוקצה בזיכרון ה-RAM.
4. רק לאחר שהנתונים נטענו בהצלחה ל-RAM, הקרנל ממשיך לשלב הבא.

שלב 6: העתקת הנתונים למרחב המשתמש (copy_to_user)

- **מה קורה בפועל:** הנתונים נמצאים כעת בזיכרון הקרנל (בתוך ה-Page Cache). אולם, תהליך המשתמש אינו יכול לגשת לזיכרון הליבה מסיבות אבטחה חמורות (הגנה על מרחב הליבה).
- הקרנל מפעיל את פונקציית ההעברה המוגנת copy_to_user. פונקציה זו מעתיקה את כמות הבייטים המבוקשת מתוך באפר הקרנל ישירות אל תוך באפר הזיכרון (buf) שהוקצה על ידי התהליך במרחב המשתמש.
- **הגנה טכנית:** אם כתובת הבאפר של המשתמש פגומה או אינה חוקית, פונקציית copy_to_user מזהה זאת ותופסת את החריגה מבלי לגרום לקריסת מערכת ההפעלה, ומחזירה שגיאת EFAULT.

שלב 7: עדכון מיקום הקובץ (f_pos)

- **מה קורה בפועל:** הקרנל מעדכן את מצביע המיקום הנוכחי של הקובץ (File Offset, f_pos בשדה בתוך אובייקט ה-struct file).
- הערך של f_pos גדל בדיוק במספר הבייטים שנקראו בפועל והועתקו למרחב המשתמש. פעולה זו מבטיחה שקריאת ה-read() הבאה תמשיך באופן לינארי מהנקודה שבה הפסיקה הקודמת. (הערה: מכיוון ש-f_pos שמור באובייקט ה-struct file של פתיחת הקובץ הספציפית, שני תהליכים שונים שפתחו את אותו קובץ יחזיקו מצביעי f_pos נפרדים ועצמאיים לחלוטין).

שלב 8: סיום והחזרה למרחב המשתמש

- **מה קורה בפועל:** הקרנל מסיר את הנעילות מעל דפי הזיכרון, מעדכן חותמות זמן של קריאה (atime ב-Inode אם נדרש), מעביר את המעבד בחזרה מ-Ring 0 ל-Ring 3, ומחזיר את מספר הבייטים שנקראו בפועל (מספר שלם חיובי) אל פונקציית ה-read() במרחב המשתמש. אם הגיעו לסוף הקובץ (EOF), הקרנל מחזיר את המספר 0.